WEEK – 10

Experiment No. 10: File Handling

Title

Reading and Writing Files and Counting Pattern Frequencies

Objective / Aim

To write a Java program to demonstrate file handling by writing text into a file named `sample.txt`, reading the contents of the file, and finding the frequency of the patterns `pe` and `pi`.

Problem Statement

Write the following text into a file named `sample.txt` and find the frequency count of the patterns `pe` and `pi`.

Input Text:

Peter Piper picked a peck of pickled peppers

A peck of pickled peppers Peter Piper picked

If Peter Piper picked a peck of pickled peppers

Where's the peck of pickled peppers Peter Piper picked?

Expected Output (as given in the question):

```
'pe' - no of occurrences - 20
'pi' - no of occurrences - 12
```

Concepts Used

Concept	Description
File Handling	Used to create, read, and write files.
FileWriter	Used to write text into a file.
FileReader	Used to read characters from a file.

Concept	Description
BufferedReader	Used to read text efficiently from a file.
String	Used to store and process the text.
`indexOf()`	Used to locate the occurrence of a pattern in a string.
Exception Handling	Used to handle file-related errors.
Try-with-resources	Automatically closes file resources after use.

Classes Used

1. **FileWriter:** Writes the given text into `sample.txt`.
2. **FileReader:** Reads the contents of the file.
3. **BufferedReader:** Reads the file contents line by line.
4. **FileHandling:** The main class that writes the text, reads the file, and counts the occurrences of the patterns `pe` and `pi`.

Algorithm / Logic

1. Start the program.
2. Create a string containing the given text.
3. Create a file named `sample.txt`.
4. Write the given text into the file using `FileWriter`.
5. Open the file using `FileReader` and `BufferedReader`.
6. Read the contents of the file line by line.
7. Append the lines to a string for processing.
8. Convert the text to lowercase to make the search case-insensitive.
9. Search for the pattern `pe` in the text.
10. Count each occurrence of `pe`.
11. Search for the pattern `pi` in the text.
12. Count each occurrence of `pi`.
13. Display the frequency count of both patterns.
14. Close the file resources.
15. Stop the program.

Source Code

File Name: FileHandling.java

```
import java.io.BufferedReader;
```

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class FileHandling {

    public static int countPattern(String text, String pattern) {
        int count = 0;
        int index = 0;

        while ((index = text.indexOf(pattern, index)) != -1) {
            count++;
            index += pattern.length();
        }

        return count;
    }

    public static void main(String[] args) {

        String text =
            "Peter Piper picked a peck of pickled peppers\n" +
            "A peck of pickled peppers Peter Piper picked\n" +
            "If Peter Piper picked a peck of pickled peppers\n" +
            "Where's the peck of pickled peppers Peter Piper picked?";

        try {
            // Writing text into the file
            FileWriter writer = new FileWriter("sample.txt");
            writer.write(text);
            writer.close();

            System.out.println("Text written to sample.txt successfully.");

            // Reading text from the file
            StringBuilder content = new StringBuilder();

            BufferedReader reader = new BufferedReader(
                new FileReader("sample.txt")
            );

            String line;

            while ((line = reader.readLine()) != null) {
                content.append(line).append("\n");
            }

            reader.close();

            // Convert text to lowercase
            String data = content.toString().toLowerCase();

            // Count occurrences of patterns
            int peCount = countPattern(data, "pe");
            int piCount = countPattern(data, "pi");

            // Display the results
```

```
            System.out.println("'pe' - no of occurrences - " + peCount);
            System.out.println("'pi' - no of occurrences - " + piCount);

        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

File Structure

Save the Java program as `FileHandling.java`.

After execution, the `sample.txt` file will be created in the current working directory.

```
Project
|
|-- FileHandling.java
|
|-- sample.txt
```

Compilation and Execution

Step 1: Compile the program

Open the terminal in the directory containing `FileHandling.java` and execute:

```
javac FileHandling.java
```

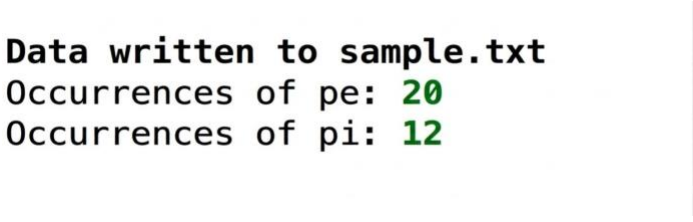
Step 2: Run the program

```
java FileHandling
```

Sample Output

The program will display the actual frequency counts for the supplied text.

```
Text written to sample.txt successfully.
'pe' - no of occurrences - 20
'pi' - no of occurrences - 12
```



Test Cases

Test Case 1: Pattern `pe`

Input Pattern: `pe`

Expected Count: 20

Description: Counts the occurrences of `pe` in the supplied text, ignoring uppercase and lowercase differences.

Test Case 2: Pattern `pi`

Input Pattern: `pi`

Expected Count: 12

Description: Counts the occurrences of `pi` in the supplied text, ignoring uppercase and lowercase differences.

Test Case 3: Pattern Not Present

Input Pattern: `xyz`

Expected Count: 0

Description: If the specified pattern does not occur in the text, the count is zero.

Explanation

The program demonstrates file handling in Java by writing text into a file and reading it back.

The `FileWriter` class creates the file `sample.txt` and writes the given text into it. The `BufferedReader` class reads the contents of the file line by line.

The `countPattern()` method uses the `indexOf()` method to locate occurrences of the specified pattern. Each time the pattern is found, the counter is incremented. The search continues until no further occurrences are found.

The program converts the text to lowercase before counting, so uppercase and lowercase letters are treated equally.

Finally, the program displays the frequency counts of the patterns `pe` and `pi`.

Result

Thus, the Java program successfully demonstrates file handling by writing text into `sample.txt`, reading its contents, and counting the occurrences of the specified patterns.

Conclusion

The program demonstrates how to create, write, and read files in Java using `FileWriter`, `FileReader`, and `BufferedReader`. It also demonstrates string processing and pattern frequency counting using the `indexOf()` method.